

Turismo Valencia RAG

Guía del proyecto desde cero

Arquitectura, recorrido de los datos, módulos, comandos y evaluación. Explicación basada en el código del proyecto revisado durante esta conversación.

La idea principal: el asistente busca información en tus documentos y utiliza un modelo de inteligencia artificial para redactar una respuesta con fuentes.

Imagina una biblioteca: los archivos son los libros, el buscador encuentra los párrafos útiles y el modelo escribe una explicación usando esos párrafos.

El diagrama completo está al final del PDF, en una página horizontal para facilitar su lectura.

1. Qué significa RAG

RAG significa generación aumentada por recuperación. El sistema recibe una pregunta, recupera fragmentos relacionados, entrega al modelo la pregunta junto con esos fragmentos y genera una respuesta basada en ellos.

Por ejemplo: «¿Qué rutas de senderismo hay en Valencia?». El buscador podría recuperar fragmentos de rutas-senderismo-valencia.txt y entregarlos al modelo para construir la respuesta. Es un ejemplo del recorrido, no el resultado de una consulta ejecutada para esta guía.

No estás entrenando al modelo con tus archivos. Construyes una biblioteca que el sistema consulta cuando recibe una pregunta.

2. Preparar la biblioteca: indexación

turismo-rag index

Paso / módulo	Qué hace
ingestion.py	Lee los archivos .txt de data/Documentos Fuente y conserva texto, fuente y categoría.
chunking.py	Divide los documentos en fragmentos de hasta 1.000 caracteres, con 150 caracteres de solapamiento.
embeddings.py	Pide a Ollama que convierta el texto en vectores mediante EmbeddingGemma.
indexing.py	Coordina la preparación y guarda textos, vectores y metadatos en ChromaDB.

El solapamiento repite una parte del texto entre fragmentos consecutivos para conservar contexto cerca de los cortes. Las medidas son caracteres, no palabras ni tokens.

Un **vector** es una lista de números que representa características del significado de un texto. Permite comparar una pregunta con los documentos aunque utilicen palabras distintas. La semejanza ayuda a buscar; no garantiza que un fragmento contenga la respuesta.

ChromaDB es la base de datos que almacena los vectores, textos y su procedencia. Sus archivos se guardan en data/vector_store/. También se escribe data/processed/chunks.jsonl: cada línea contiene un objeto JSON que describe un fragmento.

La indexación se ejecuta inicialmente y cuando cambias documentos, fragmentación o modelo de embeddings. No se repite por cada pregunta. Si el contenido y la configuración coinciden, se reutilizan los vectores existentes.

indexing.py identifica el índice mediante una huella del contenido y la configuración. Procesa los fragmentos pendientes por lotes y publica el nuevo índice solo al completar la colección. El manifiesto indica cuál colección está activa, evitando sustituirla por una preparación incompleta.

3. Arrancar la API

Una **API** es una interfaz que permite a otros programas enviar solicitudes y recibir resultados. Aquí se comunica mediante HTTP, el protocolo que también usa el navegador.

```
turismo-rag serve
```

El comando nace en pyproject.toml, donde se registra esta entrada:

```
[project.scripts]
turismo-rag = "turismo_rag.main:main"
```

Al instalar el proyecto, pip crea el comando turismo-rag. Cuando lo ejecutas, llama a la función main() de main.py. La opción serve inicia Uvicorn, que carga el objeto app de api.py.

```
uvicorn.run("turismo_rag.api:app", host=args.host, port=args.port)
```

Componente	Responsabilidad
Uvicorn	Mantiene el servidor escuchando peticiones.
FastAPI	Permite definir las rutas, las entradas y las respuestas.
api.py	Declara la aplicación FastAPI y las operaciones de este proyecto.
/docs	Interfaz interactiva para probar las peticiones desde el navegador.

api.py define la API; el comando serve arranca el servidor que permite usarla. El servidor queda escuchando en http://127.0.0.1:8000. La dirección 127.0.0.1 representa tu propio equipo.

4. El recorrido de una pregunta

Ejemplo de entrada enviada a POST /ask:

```
{
  "question": "¿Qué rutas de senderismo hay en Valencia?",
  "top_k": 5
}
```

JSON es un formato de datos con nombres y valores. top_k indica cuántos fragmentos se intentan recuperar como máximo; aquí, cinco.

Paso	Qué ocurre
1. Recibir	api.py recibe la petición en /ask.

Paso	Qué ocurre
2. Validar	schemas.py define QuestionRequest: pregunta entre 3 y 2.000 caracteres, top_k entre 1 y 10 cuando se proporciona, y categoría opcional. Rechaza campos desconocidos.
3. Coordinar	pipeline.py usa TourismAgent para ejecutar la búsqueda y después la generación.
4. Buscar	retrieval.py usa embeddings.py para vectorizar la pregunta y consulta ChromaDB. Puede filtrar por categoría y, si se configura, por distancia máxima.
5. Preparar	generation.py reúne pregunta, fragmentos e instrucciones: responder en español usando el contexto y citando fuentes.
6. Generar	La integración de LangChain llama a Ollama, que ejecuta el modelo de respuestas.
7. Devolver	Se comprueban los números de las citas y la API devuelve la respuesta estructurada.

La búsqueda compara vectores mediante distancia coseno: una distancia menor indica mayor cercanía. Esa distancia no es una probabilidad de acierto.

La respuesta incluye question (pregunta), answer (texto), sources (fragmentos recuperados), cited_sources (números citados), grounded (presencia de citas numéricamente válidas), model (modelo) y elapsed_seconds (duración). Cada fuente conserva archivo, categoría y posición dentro del documento.

Si no hay fragmentos, el generador devuelve una abstención sin llamar al modelo. Si la respuesta generada carece de citas o contiene números inexistentes, también devuelve una abstención: «No encuentro información suficiente en los documentos para responder esa pregunta».

La comprobación de citas no verifica la verdad de cada afirmación. grounded=true solo indica referencias con números válidos. Las instrucciones intentan limitar la respuesta al contexto, pero no garantizan su exactitud.

5. Ollama y los dos modelos

Ollama es el programa que ejecuta los modelos en tu equipo. El proyecto configura dos trabajos diferentes:

Modelo predeterminado	Trabajo
embeddinggemma:latest	Convierte documentos y preguntas en vectores para la búsqueda por significado.
gemma3:4b	Redacta la respuesta usando los fragmentos recuperados.

EmbeddingGemma participa al indexar y al buscar. Gemma 3 participa al generar respuestas. LangChain se utiliza en generation.py a través de ChatOllama; las llamadas de embeddings se hacen con httpx.

Ollama funciona como servicio local separado, normalmente en el puerto 11434. Tu API escucha en el 8000. ChromaDB se usa desde Python con persistencia en disco; esta implementación no exige iniciar un servidor Chroma separado.

6. Arquitectura y separación de responsabilidades

La arquitectura es una **aplicación Python modular, con flujo RAG fijo y servicios locales**. Modular significa dividir el código en archivos que tienen trabajos concretos. Las cajas del diagrama representan

responsabilidades lógicas; no representan microservicios independientes.

Parte	Módulos
Entradas	main.py recibe comandos de terminal; api.py recibe peticiones HTTP.
Coordinación	pipeline.py organiza búsqueda y generación.
Preparación	ingestion.py, chunking.py e indexing.py construyen el índice.
Búsqueda y generación	embeddings.py, retrieval.py y generation.py implementan el trabajo del RAG.
Apoyo	config.py centraliza ajustes y rutas; schemas.py define formatos y validaciones.
Calidad	evaluation.py mide resultados; tests/test_rag.py comprueba comportamientos del código.

Esta separación permite modificar, por ejemplo, las instrucciones de respuesta en generation.py sin mezclar ese cambio con la lectura de archivos.

Aunque la clase se llama TourismAgent, ejecuta una secuencia programada: buscar y responder. No hay varios agentes autónomos tomando decisiones o repartíéndose tareas. Cada pregunta se procesa de forma independiente, sin memoria de conversación.

7. Mapa de archivos y carpetas

src contiene el código fuente. Dentro, turismo_rag es el paquete que agrupa los módulos. __init__.py identifica ese paquete.

Archivo o carpeta	Para qué sirve
src/turismo_rag/main.py	Comandos index, serve, ask y evaluate.
src/turismo_rag/api.py	Aplicación FastAPI, rutas y manejo de errores.
src/turismo_rag/pipeline.py	Clase TourismAgent: coordina y mide el tiempo de la consulta.
src/turismo_rag/ingestion.py	Lectura de documentos y procedencia.
src/turismo_rag/chunking.py	División y guardado de fragmentos.
src/turismo_rag/embeddings.py	Obtención de vectores con Ollama.
src/turismo_rag/indexing.py	Creación, persistencia y publicación del índice.
src/turismo_rag/retrieval.py	Búsqueda semántica y filtros.
src/turismo_rag/generation.py	Instrucciones al modelo y control de referencias.
src/turismo_rag/schemas.py	QuestionRequest, Source y Answer: contratos de datos.
src/turismo_rag/config.py	Rutas, valores predeterminados y lectura de configuración.
src/turismo_rag/evaluation.py	Evaluación del CSV y escritura de métricas.

Archivo o carpeta	Para qué sirve
pyproject.toml	Metadatos, dependencias y registro del comando turismo-rag. TOML es un formato de configuración; no se ejecuta.
requirements-lock.txt	Versiones concretas de dependencias para reproducir el entorno.
.venv/	Entorno Python y librerías instaladas para el proyecto.
.env.example / .env	Plantilla y configuración opcional: modelos, Ollama, top_k y otros ajustes.
.gitignore	Excluye de Git entornos, configuración local y artefactos generados.
README.md	Instalación, ejecución, evaluación y documentación de uso.
data/Documentos Fuente/	Textos originales que consulta el asistente.
data/processed/chunks.jsonl	Fragmentos generados con posición y procedencia.
data/vector_store/	Base vectorial persistente y manifiesto del índice activo.
data/dataset.csv	Preguntas y referencias para evaluar; queda fuera del índice.
data/evaluation/	Resultados detallados y resúmenes de evaluación.
tests/test_rag.py	Pruebas automatizadas de componentes y casos de error.

8. Cómo se comprueba que funciona

Pruebas del código. Verifican fragmentación, entradas inválidas, citas, reutilización del índice, conservación del índice anterior ante fallos y separación de referencias de evaluación. Las pruebas unitarias no necesitan descargar modelos ni ejecutar Ollama.

```
python -m pytest -q
```

Evaluación del asistente. Usa las preguntas del CSV para medir si recupera el archivo esperado y, opcionalmente, comparar las respuestas con las referencias.

```
turismo-rag evaluate --split dev --limit 20
turismo-rag evaluate --split dev --limit 5 --generate
```

Sin --generate se evalúa la búsqueda. Con --generate también se generan respuestas. El CSV no se incorpora al índice: solo la pregunta entra al sistema y las referencias se usan después para comparar.

Las preguntas se separan de forma reproducible en desarrollo (dev) y prueba (test), aproximadamente 20 % y 80 %. Las preguntas duplicadas permanecen en la misma partición. Usa dev para ajustar y reserva test para medir el resultado final.

Métrica	Qué significa
source_hit_at_k	Proporción de consultas que recuperan el archivo esperado entre los resultados.
MRR	Premia que el archivo esperado aparezca en las primeras posiciones.

Métrica	Qué significa
F1 de palabras	Mide coincidencia de palabras entre respuesta generada y referencia.
Coincidencia exacta	Compara textos normalizados de la respuesta y la referencia.
Tiempo, errores y abstención	Miden duración, fallos y consultas sin respuesta respaldada por citas válidas.

Hit y MRR identifican el archivo, no la suficiencia del párrafo. F1 puede penalizar una paráfrasis correcta y no comprueba veracidad. Los resultados se guardan como JSONL y un resumen JSON en `data/evaluation/`.

9. Rutas disponibles en la API

Método y ruta	Función
GET /health	Comprueba índice, Ollama y modelos. Devuelve 503 si detecta problemas de disponibilidad.
POST /search	Recupera fragmentos sin generar una respuesta redactada.
POST /ask	Busca fragmentos y genera una respuesta con fuentes y citas.
GET /docs	Interfaz para probar las rutas desde el navegador.
GET /	Redirige a /docs.

Una ruta o endpoint es la dirección de una operación dentro de la API. GET suele consultar un recurso; aquí POST envía los datos de la pregunta. Los errores de validación producen HTTP 422; las dependencias no disponibles, HTTP 503; los fallos inesperados, HTTP 500.

10. Cómo ejecutarlo desde VS Code

Abre Terminal → Nueva terminal en la raíz del proyecto. Si ya instalaste el proyecto, descargaste los modelos y construiste el índice, ejecuta estas líneas una por una:

```
.\.venv\Scripts\Activate.ps1
turismo-rag serve
```

La primera activa el entorno Python del proyecto. La segunda inicia la API. Abre `http://127.0.0.1:8000/docs`, despliega POST /ask, pulsa Try it out, introduce el JSON del ejemplo y pulsa Execute. Deja la terminal abierta; Ctrl+C detiene el servidor.

Ollama debe estar activo. Si su aplicación no está abierta, ejecuta `ollama serve` en otra terminal. Para la preparación inicial, con Python 3.12, Ollama y el entorno virtual ya disponibles, el README indica:

```
python -m pip install -e ".[dev]"
ollama pull embeddinggemma:latest
ollama pull gemma3:4b
turismo-rag index
turismo-rag serve
```

Si todavía no existe `.venv`, puedes crearlo con `py -3.12 -m venv .venv` y luego activarlo. La descarga inicial de dependencias y modelos requiere internet. Las consultas usan los recursos de tu equipo.

También puedes preguntar desde la terminal sin arrancar la API:

```
turismo-rag ask "¿Qué puedo visitar en la Albufera?"
```

Ese comando utiliza el mismo flujo de búsqueda y generación sin pasar por HTTP. Si no activas el entorno, después de instalar el proyecto puedes iniciar la API con:

```
.\.venv\Scripts\python.exe -m turismo_rag.main serve
```

Los valores de config.py funcionan sin .env. Si necesitas cambiarlos, copia .env.example a .env, edítalo y reinicia la API. Si cambias los documentos, la fragmentación o el modelo de embeddings, vuelve a indexar con la API detenida.

11. Orden recomendado para estudiar el código

Para seguir una pregunta: main.py → api.py → pipeline.py → retrieval.py → generation.py. Consulta schemas.py para entender los datos de entrada y salida, y embeddings.py para la conversión de texto a vectores.

Para seguir la preparación: ingestion.py → chunking.py → embeddings.py → indexing.py. Después revisa config.py, evaluation.py y las pruebas.

El diagrama de la siguiente página resume los tres recorridos: azul para preparar documentos, verde para responder preguntas y naranja para evaluar. Fue generado con la herramienta integrada image_gen; el prompt se conserva en docs/arquitectura-imagen-prompt.txt.

Diagrama general de arquitectura



Turismo Valencia RAG · Arquitectura del proyecto



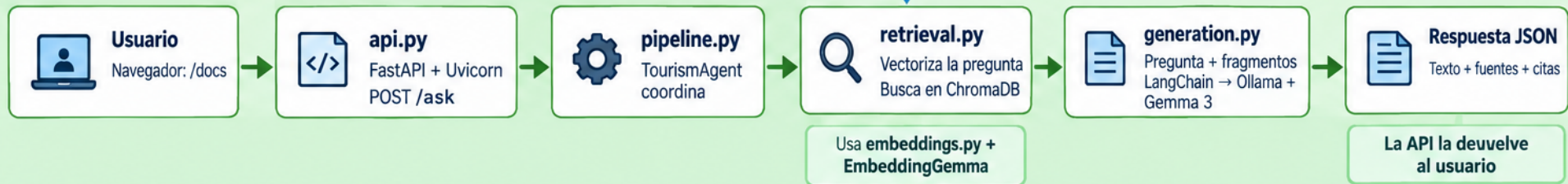
1. PREPARAR LOS DOCUMENTOS · turismo-rag index

indexing.py coordina la preparación



2. RESPONDER PREGUNTAS · turismo-rag serve

Fragmentos recuperados



3. EVALUAR · turismo-rag evaluate



main.py: comandos de terminal | config.py + .env: ajustes | schemas.py: formatos y validación | tests/test_rag.py: pruebas | pyproject.toml: instalación y dependencias

Aplicación Python modular y local. Flujo RAG fijo, sin memoria de conversación.
Ollama ejecuta dos modelos: uno para vectores y otro para redactar.